

Supplementary Material B

HypatiaX: Noise Robustness, Sample Complexity, and Convergence

of Hybrid LLM–Neural Symbolic Regression on a Custom 30-Equation Benchmark

Ruperto Pedro Bonet Chaple

ruperto.bonet@modelphysmat.com

Independent Researcher, London, United Kingdom

Editor:

Abstract

We present an extended evaluation of the HypatiaX symbolic regression framework on a custom 30-equation benchmark, integrating a six-method noiseless comparison with a comprehensive noise robustness sweep ($\sigma \in \{0, 0.05, 0.1, 0.5, 1\}\%$) and a sample complexity sweep ($n \in \{50, 100, 200, 500, 750, 1000\}$), focusing on the two top-performing core methods: EHSDEFI (M3) and HLLMNN (M4).

Following correction of a measurement harness bug in HLLMNN (see Section 10), both methods achieve median $R^2 = 1.000000$ across all tested conditions. EHSDEFI achieves 100% recovery at all *noisy* conditions ($\sigma > 0$) but only 76.7% (23/30) at $\sigma = 0\%$ under the strict noiseless threshold ($R^2 \geq 0.999999$; Table 9).

HLLMNN achieves 100% recovery at all *noisy* conditions ($\sigma > 0$) and 100% (30/30 equations, v2-corrected) under the strict noiseless protocol ($R^2 > 0.9999$). HDSv50_2 achieves 70.0% (21/30 equations) under the same noiseless protocol.

All results in this document use the v2 corrected harness; v1 figures are retained in the repository commit history for audit purposes only. Both reach median $R^2 = 1.000000$ at $n = 50$ samples (noiseless sample-complexity sweep; noisy $n > 50$ runs are not available), though EHSDEFI’s recovery rate dips to 96.7% at larger n (Table 11). EHSDEFI offers exact symbolic interpretability; HLLMNN is $\sim 1.1\text{--}1.2\times$ faster at every noise level .

On this 30-equation custom benchmark (**not** a Feynman subset — see Table 4), EHSDEFI achieves 76.7% recovery and HDSv50_2 achieves 70.0%, alongside AI Feynman 2.0’s published 79.3% on the unrelated 100-equation Feynman/SRBench suite . The AI Feynman figure is shown for general SOTA context only: published baselines were evaluated on all 100 Feynman equations under the standardised SRBench protocol; these figures should not be read as a general state-of-the-art claim. We additionally correct four methodological issues: undisclosed hardcoded PURELLM results, scale-biased raw RMSE, missing Wilcoxon significance tests, and mismatched pass-rate thresholds.

Keywords: symbolic regression, scientific discovery, large language models, neural networks, equation-recovery benchmark, noise robustness, sample complexity, hybrid systems, reproducibility

1 Introduction

This document is Supplementary Material B to the main JMLR submission *HypatiaX: A Hybrid Symbolic-Neural Framework for Extrapolation-Reliable Analytical Discovery* (`jmlr_paper_main.tex`).

Cross-reference note (v5, updated 2026-07-28). The main paper has been updated since this supplementary was written, and again since the v4 cross-reference below was added. Readers are directed to the following sections for context:

- **The main paper’s *Protocol-30 Extrapolation Benchmark* ($n = 30$) subsection (Results section)** — the main paper reports the 2026-08-12 results: **21/30 (70.0 %)** for HybridDiscoverySystem v50_2 under the random 80/20 split and **18/30 (60.0 %)** under the PCA-directed 40%/60% split, at the strict $R^2 \geq 0.999999$ threshold on the method-reported R^2 (13/30 on the held-out 60% for the PCA split). The same runs give PureLLM 29/30 on both splits (authors’ recount; 18/19 on the 19 live rows), a per-equation pass count on the same 30 equations. Counts for the other methods in the six-method `benchmark_results.json` are not reported here; that file is a different run from the 170344 file used elsewhere in this document.
- **The main paper’s *Nguyen-12 Benchmark (Standard SR Suite)* subsection** — reports PySR-only and HypatiaX on the standard Nguyen-12 suite under an axis-aligned extrapolation protocol. **Figures revised 2026-09-28** and rebuilt from the consolidated CI result files: seed 42 gives 10/12 for both systems, and across five seeds HypatiaX recovers 51/60 (85.0%) and PySR-only 49/60 (81.7%) at $R^2 \geq 0.9999$. HypatiaX fails Nguyen-12 on all five seeds (LLM gate rejects all candidates; the run aborts before any search). The earlier 8/12 P / 7/12 H figures, the 11/12 / 10/12 figure, the 4/12 caption figure and the Neural MLP (0/12) figure, all cited here in earlier versions, are retracted: none reproduces from any current source file, and no Neural MLP was ever run for this benchmark. See the main paper (the Nguyen-12 table, tablenote ‡, and the “Nguyen-12 Benchmark (Standard SR Suite)” section). The pipeline uses per-equation axis-aligned variable ranges, not PCA-directed ordering.
- **Section 7.4 Proposition 1 (Routing Cascade Convergence)** — formal guarantee that Phase 2 warm-starting is monotonically non-degrading, complementing the routing improvements described in Supplementary A.

The noise robustness, sample complexity, and convergence analyses in this document remain valid and unchanged.

Symbolic regression (SR) aims to discover closed-form analytical expressions from data without prior knowledge of the functional form. The Feynman SR Benchmark (Udrescu and Tegmark, 2020) provides a canonical evaluation suite of 100 physics equations (the standard set used under SRBench).

This supplementary integrates two complementary evaluations:

1. A **six-method noiseless comparison** (corrected from the companion report) establishing the state-of-the-art context. **[Note: this evaluation uses a custom 30-equation benchmark, not a subset of the Feynman suite described above. Its 11 domains are biology, chemistry, electrochemistry, electromagnetism,**

electrostatics, magnetism, mechanics, optics, probability, quantum, and thermodynamics; several of these (biology, chemistry, electrochemistry, probability) are outside the Feynman benchmark’s scope. The domain list above is taken from the source files. See Section 5 and the source-file note on Table 4.]

2. A **sweep evaluation** (v2, post bug-fix, 16 March 2026) covering EHSDEFI (M3) and HLLMNN (M4) across noise levels and sample sizes, with convergence, win-rate, and architecture analysis.

Version note. A v1 sweep (15 March 2026) incorrectly reported HLLMNN as having a persistent catastrophic failure on Newton’s gravitational force ($R^2 \approx 0.42\text{--}0.87$). This was caused by a measurement bug in `evaluate_llm_formula` (Section 10), not an architectural limitation. All results in this paper are from the v2 rerun unless explicitly labelled v1.

2 Related Work

SR has evolved from genetic programming (Schmidt and Lipson, 2009) through physics-inspired decomposition (Udrescu and Tegmark, 2020) to transformer-based generation (Biggio et al., 2021; Shojaei et al., 2023) and deep RL over expression trees (Petersen et al., 2021). The best-published result on the noiseless Feynman benchmark prior to this work is AI Feynman 2.0 at 79.3% (Udrescu et al., 2020).

Benchmark integrity has received increasing attention. Satvaty et al. (2024) show that LLMs can reproduce iconic equations from training data rather than inferring them from observations, inflating reported pass rates. The hardcoding audit applied to PURELLM in Section 3.3 follows this line of work.

A methodological concern specific to tiny-scale physics equations is that benchmark harnesses using absolute sum-of-squares thresholds silently misclassify correct formulas. The fix described in Section 10 is a general contribution to SR benchmarking practice.

3 Methods

3.1 Six-Method Benchmark Suite

3.2 Sweep Evaluation Design (M3 vs. M4)

Hyperparameters: NN seeds = 5, random seed 42, method timeout = 900 s, PySR timeout = 1100 s, threshold (noise sweep) = 0.999999 at $\sigma = 0$, 0.995 at $\sigma \in \{0.05, 0.1, 0.5\}\%$ and 0.99 at $\sigma = 1\%$ (sample-complexity sweep, noiseless: 0.999 at $n = 50$, 0.9995 at $n = 100$, 0.9999 at $n \geq 200$). All 30 equations completed with zero timeouts across all 11 conditions.

3.3 Integrity Annotation: Hardcoded Results

Following Satvaty et al. (2024), each PURELLM noiseless row is audited for whether the language model was called. The `is_hardcoded` flag is set when the baseline returns `method="pure_llm_hardcoded"`, i.e. when `_try_hardcoded_formula` in `baseline_pure_llm_defi_discovery.py` returns a formula without any LLM call. It is a record of that code path, not a statistical memorisation test. Two kinds of bypass exist: eight equations return the benchmark’s

Table 1: Methods under evaluation: six-method noiseless benchmark.

Method	Type	Description
PURELLM	Core	Zero-shot LLM formula inference. No numerical fitting. ! 11/30 results flagged hardcoded (Section 3.3).
EHSDEFI (M3)	Core	Ensemble-first system (140/150 = 93.3% of routing decisions; Table 13) combining LLM-guided generation, NN ensemble scoring, and DeepFit optimisation.
HLLMNN (M4)	Core	LLM-first + NN residual correction (123/150 = 82.0% LLM-first routing decisions; Table 13); $\sim 1.1\text{--}1.2\times$ faster than M3 (Table 10).
IMPNN	Core	Pure neural baseline ($1 \rightarrow 128 \rightarrow 64 \rightarrow 32 \rightarrow 1$, 3 seeds).
SYMLLM	Tools	LLM-guided symbolic search with CAS verification.
HDSv50.2	Tools	HybridDiscoverySystem v50.2; 21/30 pass at 75.3 s mean runtime in the noiseless protocol.

Table 2: Sweep design.

Sweep	Axis varied	Fixed	Conditions
Noise	$\sigma \in \{0, 0.05, 0.1, 0.5, 1\}\%$ *	$n = 200^\dagger$	5 levels
Sample size	$n \in \{50, 100, 200, 500, 750, 1000\}$	$\sigma = 5\%$	6 sizes

*Noise levels as recorded in the sweep source files; see Table 8.

own ground-truth expression (GT in Table 3), and three use a fixed functional form whose parameters are fitted to the data by least squares (Fit). A twelfth branch (Arrhenius) did not fire. We call the remaining 19 rows *live*. The same 11 equations are identified in the eleven per-domain `protocol_core_noiseless_20260812_*.json` files (by flag), in the consolidated `benchmark_results.json` and in `benchmark_results_pca_4060.json` (by timing, 0.5–1.6 ms, since those two flat files carry no flag), using `recount_hardcoded.py`. The 170344 file was not part of the recount. 11 of 30 PURELLM noiseless rows bypassed the model (Table 3).

3.4 Evaluation Metrics

Coefficient of determination (R^2). $R^2 = 1 - \text{SS}_{\text{res}}/\text{SS}_{\text{tot}}$.

Normalised RMSE ($\widetilde{\text{RMSE}}$). $\widetilde{\text{RMSE}} = \text{RMSE}/\sigma_y = \sqrt{1 - R^2}$. Raw RMSE is dominated by large-scale equations; median $\widetilde{\text{RMSE}}$ or R^2 should be used for cross-equation comparison.

Table 3: Equations for which PURELLM bypassed the language model (flagged `is_hardcoded=True`). GT: the benchmark’s ground-truth expression is returned; Fit: fixed form with least-squares parameters fitted to the data. R^2 as recorded in the recount; every bypassed row passes.

Equation	Domain	Type	R^2 (NL)
Michaelis–Menten enzyme kinetics	Biology	Fit	1.0000000
Logistic growth rate	Biology	Fit	1.0000000
Allometric scaling law	Biology	Fit	1.0000000
Nernst equation (electrode potential)	Electrochemistry	GT	1.0000000
Clausius–Mossotti effective field	Electrostatics	GT	1.0000000
Lorentz force: $F = qvB$	Magnetism	GT	1.0000000
Gaussian/normal distribution PDF	Probability	GT	1.0000000
Zeeman energy (electron spin)	Quantum	GT	1.0000000
Bose–Einstein occupation number	Quantum	GT	1.0000000
Planck blackbody spectral radiance	Thermodynamics	GT	1.0000000
Fourier’s law of heat conduction	Thermodynamics	GT	1.0000000

Recovery rate. Fraction of equations achieving $R^2 \geq \text{threshold}$ (≥ 0.9999 noiseless; ≥ 0.995 noisy). Catastrophic = $R^2 < 0.90$.

Statistical tests. Wilcoxon signed-rank tests for all pairwise comparisons, Bonferroni corrected ($\alpha^* = 0.05/15 \approx 0.0033$ for $\binom{6}{2} = 15$ pairs).

4 Algorithm

Algorithm 1 HypatiaX hybrid symbolic discovery (M3/M4 core loop)

Require: Dataset $\mathcal{D}$, LLM proposer $\mathcal{M}$, NN ensemble f_θ , threshold τ

Ensure: Discovered expression $\hat{e}$

```

1:  $d \leftarrow \text{DESCRIBE}(\mathcal{D})$ 
2:  $\{e_1, \dots, e_K\} \leftarrow \mathcal{M}(d, \mathcal{D})$ 
3: for each candidate  $e_k$  do
4:   Fit  $\theta^* \leftarrow \arg \min_{\theta} \sum_i (f_{\theta}(e_k, x_i) - y_i)^2$ 
5:   Compute  $R_k^2$  with relative threshold (Section 10)
6:   if  $e_k$  invalid or numerically unstable then
7:     discard
8:   end if
9: end for
10: M3 path:  $\hat{e} \leftarrow \arg \max_{e_k} R_k^2$ 
11: M4 path: if  $R_{\text{LLM}}^2 \geq \tau$ , return LLM formula; else apply NN residual correction
12: return  $\hat{e}$ 

```

5 Six-Method Noiseless Results

Table 4: Six-method aggregate performance, noiseless protocol ($R^2 \geq 0.999999$, 30 equations). **!** PURELLM pass rate includes 11/30 hardcoded results. Source: `protocol_core_noiseless_20260812.170344.json` for all rows except the PURELLM pass count. The PURELLM row is a recount over the eleven per-domain `protocol_core_noiseless_20260812.*.json` files: 29/30 over all rows, of which 11 are bypassed rows (Table 3) that all pass, and 18/19 on the *live* rows where the model was called; the 170344 file reports 30/30 (the one-test difference is Curie’s law). This is a custom benchmark, not the Feynman-30 suite (Section 1); its 11 domains are biology, chemistry, electrochemistry, electromagnetism, electrostatics, magnetism, mechanics, optics, probability, quantum, and thermodynamics.

Method	Type	Avg		
		Pass Rate	Mean R^2	Runtime
		(valid tests)		
PURELLM ! [†]	Core	29/30 (96.7%) !	—	4.7 s
EHSDEFI (M3)	Core	23/30 (76.7%)	0.999998	7.6 s
HDSv50_2	Tools	21/30 (70.0%)	0.999473	75.3 s
HLLMNN (M4)	Core	30/30 (100.0%)	1.000000	6.9 s
SYMLLM	Tools	28/30 (93.3%)	0.999711	39.1 s
IMPNN	Core	0/30 (0.0%)	0.999222	2.5 s

[†] Recount: 29/30 over all rows, 18/19 live rows, 11/11 bypassed rows. Mean R^2 and runtime were not recomputed; the 4.7 s are

5.1 Context Comparison with Published SR Systems

5.2 Scale-Invariant RMSE

After replacing raw RMSE with $\widetilde{\text{RMSE}} = \sqrt{1 - R^2}$, four of six methods cluster in mean $\widetilde{\text{RMSE}} \in [0.000, 0.021]$; PURELLM is the outlier on mean ($\widetilde{\text{RMSE}}_{\text{mean}} = 0.1042$) and also on max ($\widetilde{\text{RMSE}}_{\text{max}} = 2.0063$, driven by the negative- R^2 “Thin Lens” failure disclosed in Table 4). PURELLM and HDSv50_2 each have raw RMSE values of `inf/nan/sentinel`-overflow in the source data for specific equations (see table note), so their mean raw RMSE over all 30 equations is not meaningful; $\widetilde{\text{RMSE}}$ remains well-defined for every equation with a finite R^2 since it is derived directly from R^2 .

Table 5: Performance of this work’s methods on the 30-equation custom noiseless benchmark (**not** a Feynman subset — see Section 1 and the source-file note on Table 4), shown alongside published SR systems’ figures on the unrelated, standard 100-equation Feynman/SRBench suite for general context only. PURELLM excluded (Section 3.3). **Note:** the published baselines (AI Feynman, NeSymReS, TPSR, DSR) and this work’s methods were **not** evaluated on the same equation set; the “vs AI Feynman” deltas below are illustrative only and should not be read as a head-to-head benchmark result.

System	Type	Pass Rate	Note
EHSDeFi (this work)	Hybrid	76.7%	−2.6 pp vs AI Feynman (different sets)
HDSv50_2 (this work)	Hybrid	70.0%	−9.3 pp vs AI Feynman (different sets)
AI Feynman 2.0	Symbolic	79.3%	Udrescu et al. (2020) (100-eq, SRBench)
NeSymReS	Neural	59.4%	Biggio et al. (2021) (100-eq, SRBench)
TPSR	Transformer	56.0%	Shojaee et al. (2023) (100-eq, SRBench)
DSR	Deep SR	32.0%	Petersen et al. (2021) (100-eq, SRBench)

5.3 Wilcoxon Significance Tests

6 Sweep Results: Noise Robustness (M3 vs. M4)

Table 8: R^2 statistics per noise level ($n = 200, 30$ equations).

σ	EHSDeFi (M3)			HLLMNN (M4)		
	Median	Min	Std	Median	Min	Std
0%	1.000000	0.999979	4.8×10^{-6}	1.0000000	1.000000	0.0
0.05%	0.9999998	0.997348	0.0010	0.9999998	0.997807	0.0006
0.1%	0.9999998	0.992748	0.0016	0.9999998	0.997810	0.0006
0.5%	0.9999998	0.997504	0.0010	0.9999998	0.997789	0.0006
1%	0.9999998	0.997325	0.0010	0.9999998	0.997720	0.0007

Key observations. The tables in this section (8, 9, 10) are computed from the raw noise-sweep source files (`noise_sweep_20260812_155519_nshards03.json`, `..._155520_nshards05.json`, `..._155532_nshards01.json`, `..._155541_nshards02.json`, `..._155544_nshards04.json`), cross-checked against their raw per-test companions `protocol_core_noiseless_20260812_170344.json` and four `protocol_core_noisy_*` files with matching timestamps to the second. The noise

Table 6: Raw RMSE vs. normalised RMSE ($\widetilde{\text{RMSE}} = \sqrt{1 - R^2}$), 30 equations. Regenerated from the eleven per-domain `protocol_core_noiseless_20260812_*.json` files (not the 170344 file that supplies five rows of Table 4; see its tablenote).

Method	Mean RMSE	Mean NRMSE	Max NRMSE
PURELLM(core)	0.0876 [†]	0.1042	2.0063
EHSDEFI(core)	0.5672	0.0037	0.0735
HDSv50.2(tools)	2,740.0 [‡]	0.0199	0.1675
SYMLLM(tools)	106.7	0.0099	0.2091
IMPNN(core)	361.4	0.0209	0.2419
HLLMNN(core)	0.0002	0.0002	0.0038

[†] Two equations (“Kinetic Energy,” “Logistic Growth”) have raw RMSE = ∞ in the source file, from the catastrophic/truncated failures disclosed in Table 4, and are excluded from this mean. Over the 28 equations with finite RMSE, mean RMSE ≈ 0.0876 .

[‡] Three equations (Coulomb’s Law, Lorentz Force, Diffraction) have raw RMSE=`nan` in the source file despite good R^2 on the same equations, and one (Bernoulli’s Equation) reports a 10^{150} sentinel/overflow value; all four are excluded from this mean. Over the 26 remaining equations, mean RMSE $\approx 2,740$. This RMSE-specific irregularity is unexplained and is noted for the record rather than resolved here.

levels recorded in these files are $\sigma \in \{0, 0.05, 0.1, 0.5, 1\}\%$. M3 exhibits a std of ~ 0.001 – 0.0016 across $\sigma > 0$, higher than M4’s ~ 0.0006 – 0.0007 . M4 shows zero-variance convergence at $\sigma = 0\%$: all 30 equations reach $R^2 = 1.000000$ exactly, because the LLM-first path generates the exact formula and the NN residual correction drives the fit to machine precision. **M3 does not show the same noiseless convergence:** at $\sigma = 0\%$ M3’s strict recovery rate (threshold $R^2 \geq 0.999999$, per the source file’s own recorded `threshold_used`) is **76.7% (23/30)**, not 100% — see §9 below.

Table 9: Recovery rate and catastrophic failure count per noise level ($n = 200$). Recovery uses each shard’s recorded threshold: $R^2 \geq 0.999999$ at $\sigma = 0\%$, $R^2 \geq 0.995$ at $\sigma = 0.05$, 0.1 and 0.5% , and $R^2 \geq 0.99$ at $\sigma = 1\%$, so rates are not on a single scale across rows.

σ	M3 Recovery	M3 Catastrophic	M4 Recovery	M4 Catastrophic
0%	76.7% [‡]	0	100.0%	0
0.05%	100.0%	0	100.0%	0
0.1%	96.7%	0	100.0%	0
0.5%	100.0%	0	100.0%	0
1%	100.0%	0	100.0%	0

[‡] Source file `noise_sweep...nshards01.json`, $\sigma = 0$, `threshold_used`=0.999999: 23/30. M3 is *not* perfect on noiseless recovery.

Table 7: Wilcoxon signed-rank tests on noiseless R^2 , all pairwise comparisons (two-sided, 30 equations; n_{eff} is the number of non-zero paired differences, Bonferroni $\alpha^* = 0.0033$). $**\alpha^*$; $*\alpha = 0.05$ only; n.s. not significant. Recomputed from the eleven per-domain noiseless files (the source of the PURELLM recount in Table 4; its other rows come from the 170344 file); invalid/`nan` R^2 values are treated as 0 for the paired test. n_{eff} is below 30 where methods tie.

Method A	Method B	Statistic	n_{eff}	p -value	Sig.
PURELLM	IMPNN	99.0	30	0.0050	*
PURELLM	EHSDEFI	112.0	30	0.0120	*
PURELLM	HLLMNN	2.0	7	0.0422	*
PURELLM	SYMLLM	87.0	21	0.3219	n.s.
PURELLM	HDSv50_2	90.0	23	0.1443	n.s.
IMPNN	EHSDEFI	52.0	30	0.0001	**
IMPNN	HLLMNN	0.0	30	< 0.001	**
IMPNN	SYMLLM	78.0	30	0.0010	**
IMPNN	HDSv50_2	146.0	30	0.0767	n.s.
EHSDEFI	HLLMNN	74.0	30	0.0007	**
EHSDEFI	SYMLLM	154.0	30	0.1094	n.s.
EHSDEFI	HDSv50_2	196.0	30	0.4645	n.s.
HLLMNN	SYMLLM	58.0	22	0.0262	*
HLLMNN	HDSv50_2	43.0	24	0.0022	**
SYMLLM	HDSv50_2	81.0	24	0.0487	*

Notes. (i) Pairs with $n_{\text{eff}} = 30$ have exact p -values; pairs with $n_{\text{eff}} < 30$ use the normal approximation, which is unreliable at $n_{\text{eff}} = 7$. (ii) Three starred results are sensitive to the approximation: with a continuity correction, PURELLM vs. HLLMNN gives $p = 0.052$ and SYMLLM vs. HDSv50_2 gives $p = 0.050$ (both would be not significant at $\alpha = 0.05$), while HLLMNN vs. SYMLLM stays at $p = 0.027$.

None of the three survives the Bonferroni correction. (iii) PURELLM rows include the 11 bypassed equations (Table 3); comparisons involving PURELLM therefore do not measure an unaided language model.

Table 10: Average per-equation computation time (seconds) per noise level. Every cell is the per-equation `time` field averaged across all 30 tests in the five matching raw runner logs: `protocol_core_noiseless_20260812_170344.json` ($\sigma = 0\%$) and `protocol_core_noisy_20260812_173425.json` ($\sigma = 0.05\%$), `..._172958.json` ($\sigma = 0.1\%$), `..._173523.json` ($\sigma = 0.5\%$), and `..._175318.json` ($\sigma = 1\%$). The maximum M3 value across all 150 equation-runs (5 files $\times$ 30 equations) is 7.77 s.

σ	M3 avg (s)	M4 avg (s)	Speedup
0%	7.55	6.92	1.1 $\times$
0.05%	7.66	6.55	1.2 $\times$
0.1%	7.44	6.46	1.2 $\times$
0.5%	7.77	6.56	1.2 $\times$
1%	7.01	6.33	1.1 $\times$

Note on M3 noiseless cost. The source file for $\sigma = 0\%$ records a mean M3 time of 7.55 s, in line with every other noise level in Table 10; none shows an exhaustive-search spike. The per-equation averages were re-derived from the raw runner logs (see the note on Table 10) and agree with the table to two decimal places at all five noise levels. Across all five real noise levels, M3 and M4 have comparable, stable per-equation runtimes (M3: 7.0–7.8 s; M4: 6.3–6.9 s), with M4 consistently ~ 1.1 – $1.2\times$ faster than M3.

7 Sweep Results: Sample Complexity (M3 vs. M4)

Table 11: R^2 and RMSE per sample size (noiseless, 30 equations; recovery thresholds used by the runner: 0.999 at $n = 50$, 0.9995 at $n = 100$, 0.9999 at $n \geq 200$). All six sample-size shards (`sample_complexity_20260812.*nshards01--06.json`, $n = 50, 100, 200, 500, 750, 1000$, all `mode: noiseless`) were checked cell by cell against each shard’s `per_equation` data.

n	EHSDeFi (M3)			HLLMNN (M4)		
	Med R^2	Min R^2	Med RMSE	Med R^2	Min R^2	Med RMSE
50	1.000000	0.9982	0.0006	1.0000000	1.0000	0.0000
100	1.000000	0.9997	0.0011	1.0000000	1.0000	0.0000
200	1.000000	1.0000	0.0009	1.0000000	1.0000	0.0000
500	1.000000	0.9999	0.0016	1.0000000	1.0000	0.0000
750	1.000000	0.9997	0.0021	1.0000000	1.0000	0.0000
1000	1.000000	0.9999	0.0013	1.0000000	1.0000	0.0000

HLLMNN is exact (min $R^2 = 1.0000$, RMSE= 0.0000) at every tested sample size. EHS-DEFI reaches median $R^2 = 1.000000$ by $n = 50$, but its *recovery rate* (fraction ≥ 0.9999) is 96.7% (29/30) at $n = 50$, reaches 100% at $n = 100$ and $n = 200$, then dips back to 96.7% at $n = 500, 750$, and 1000 — i.e. one equation stays just below threshold at large n rather than the metric monotonically improving with sample size. “Both converge at $n = 50$ ” is accurate for *median* R^2 but overstates EHSDEFI’s recovery-rate behaviour.

8 Convergence Analysis

M3 shows stable convergence: R^2 std ≤ 0.001 for $\sigma > 0$. M4 exhibits zero-variance convergence (std(R^2) = 0.000) at all noise levels in v2.

Both methods reach median $R^2 = 1.000000$ by $n = 50$ in the noiseless sweep (all six sample-size shards now verified; Table 11).

Convergence boundary. The following conditions are needed for comprehensive generalisation claims: (i) higher-dimensional inputs (≥ 6 variables); (ii) PDE-form targets; (iii) extension to the full AI Feynman dataset (> 100 equations).

9 Win Rate and Architecture Analysis

Table 12: Head-to-head win rates (M3 vs. M4) on R^2 . Noise sweep: 150 paired comparisons (5 noise levels $\times$ 30 equations; pairs matched by shard and test description). Sample-complexity (SC) sweep: 180 paired comparisons (6 sample sizes $\times$ 30 equations; pairs matched by sample size and equation, from the six `sample_complexity_*.csv` shards). Regenerated with `compute_winrate.py`. A pair is *tied* if both systems have $R^2 > 0.9999$ (the paper’s original tie rule); otherwise the higher R^2 wins.

Outcome	Noise sweep	SC sweep
M3 strictly higher R^2	0/150 (0.0%)	0/180 (0.0%)
M4 strictly higher R^2	28/150 (18.7%)	8/180 (4.4%)
Tied (both $R^2 > 0.9999$)	122/150 (81.3%)	172/180 (95.6%)

Sensitivity to the tie rule. The split depends strongly on how ties are defined (noise sweep, $n = 150$). Under exact floating-point comparison M3 is nominally higher on 88 pairs and M4 on 62, but most of these differences are below 10^{-9} and reflect numerical noise rather than a real accuracy gap. With an absolute tolerance of 10^{-6} , M3 is higher on 0, M4 on 35 (23.3%) and 115 are tied (76.7%); at 10^{-4} the figures are 0 / 23 (15.3%) / 127 (84.7%); at 10^{-3} they are 0 / 19 (12.7%) / 131 (87.3%). Under no tolerance of 10^{-6} or looser does M3 exceed M4 on any pair, and M4 is strictly higher on 12.7–23.3% of pairs depending on the tolerance. M4 therefore never loses to M3 beyond numerical noise on the noise sweep, and most comparisons are ties.

The sample-complexity sweep ($n = 180$) shows the same pattern. Under exact comparison M3 is higher on 0 pairs, M4 on 179 and 1 is tied; with an absolute tolerance of 10^{-6} the figures are 0 / 38 (21.1%) / 142 (78.9%); at 10^{-4} they are 0 / 8 (4.4%) / 172 (95.6%); at 10^{-3} they are 0 / 1 (0.6%) / 179 (99.4%). M4’s 8 strict wins under the $R^2 > 0.9999$ rule fall at $n = 50$ (2), $n = 100$ (3), $n = 200$ (0), $n = 500$ (1), $n = 750$ (1) and $n = 1000$ (1). M3 is never higher than M4 under any tie rule, including exact comparison.

Table 13: Internal routing statistics — evidence that M3 and M4 are genuinely distinct architectures. Regenerated by `generate_tables.py` (`arch.tex`) from the five `protocol_core` result files (one noiseless and four noisy runs, $\sigma \in \{0, 0.05, 0.1, 0.5, 1\}\%$; 5 noise levels $\times$ 30 equations = 150 records per system). “Ensemble-first” (M3) and “LLM-first” (M4) denote the share of records whose recorded `metadata.decision` routed to that branch; this is a routing share, not a solve rate, and the `success` flag is not used. The pool mixes one noiseless run ($\sigma = 0$) with four noisy runs. These counts supersede the earlier 92.7%/74.7% figures (139/150, 112/150), which could not be reproduced from any available result file under any routing, `nn_applied`, or R^2 -threshold rule tried.

Decision path	M3 (EHSD _{Fi})	M4 (HLLM _{NN})
Primary path (routing share)	140/150 (93.3%) ensemble-first	123/150 (82.0%) LLM-first
Alternative path	10/150 (6.7%) NN	27/150 (18.0%) ensemble
Secondary path	NN refinement	NN residual correction
Architecture type	Ensemble-first	LLM-first + NN-refinement

Table 14: Routing by noise level (30 equations per level), same source as Table 13 (`arch_by_noise.tex`). M3 routes identically at every level; M4 routes all noiseless records to the LLM but 20–27% of noisy records to the ensemble.

σ (%)	M3 ensemble	M3 NN	M4 LLM	M4 ensemble
0	28	2	30	0
0.05	28	2	22	8
0.1	28	2	24	6
0.5	28	2	23	7
1	28	2	24	6
Total	140	10	123	27

When to prefer M3 vs. M4.

- **Prefer M3** when exact closed-form symbolic expressions are required, time is not a constraint, or noiseless exact symbolic recovery is the primary objective.
- **Prefer M4** when wall-clock time is a factor (M4 is ~ 1.1 – $1.2\times$ faster than M3; §6), near-zero RMSE is prioritised, or $\sigma > 0$ conditions are expected.

10 Root-Cause Analysis: Newton’s Gravity Measurement Bug

10.1 Symptom

M4 reported $R^2 \in [0.42, 0.87]$ on Newton’s gravitational force across all 11 tested conditions in v1, despite the LLM generating the correct formula on every run.

10.2 Root Cause

`evaluate_llm_formula` used:

```
r2 = 1 - (ss_res / ss_tot) if ss_tot > 1e-10 else 0
```

For Newton’s gravity at tested parameter ranges, outputs are $\sim 10^{-11}$ N, giving $ss_tot \approx 1.8 \times 10^{-18}$ — eight orders of magnitude below the threshold. The function always returned $r2 = 0.0$, triggering NN fallback.

10.3 Fix Applied

```
_tol = 1e-10 * float(np.max(np.abs(y_true))**2) * len(y_true)
r2    = 1 - (ss_res / ss_tot)
        if ss_tot > max(_tol, 1e-300) else 0
```

The same fix was applied at three code locations: `evaluate_llm_formula` (line 462), `train_nn` (line 384), and the ensemble blend R^2 calculation (line 700).

10.4 Additional Fixes in the Same Commit

- `force_llm` flag: now correctly bypasses NN training.
- `PureLLMBaseline`: validates non-empty `python_code`.
- Training epochs: corrected from 300 to 1000 (scheduler requires ≥ 1000).
- Log-space guard: unconditionally enables log-space training when `y_range > 3.0` decades.

General contribution. Any SR benchmark harness using absolute sum-of-squares thresholds will silently misclassify correct formulas on equations with output values far from unity. The relative threshold fix should be adopted as standard practice.

11 Discussion

11.1 Hardcoding and LLM Benchmark Integrity

Of PURELLM’s 30 noiseless rows, 11 (36.7%) did not call the language model: 8 return the benchmark’s own ground-truth expression and 3 use a fixed form with parameters fitted to the data (Table 3). In the 170344 file, EHSDEFI’s returned formula is character-for-character identical to PURELLM’s on all 11 of these bypassed rows, and on only 2 of the 19 live rows. The cause is in the code: EHSDEFI delegates formula generation to `PureLLMBaseline.generate_formula`, which tries the hardcoded formulas before any language-model request; EHSDEFI’s pass count on the bypassed rows (6 of 11, against 17 of 19 on live rows) also depends on its NN and ensemble logic. The all-domains LLM+NN hybrid delegates in the same way, but its stored output has no formula, so whether the bypass fired for it cannot be checked from the JSON. PURELLM passes 29 of 30 rows (96.7%) overall, 11 of 11 bypassed rows and 18 of 19 (94.7%) *live* rows, i.e. rows where the model was actually called. The live figure is not an unbiased estimate of unaided performance: according to comments in the baseline source, the bypass branches were added for equations on which the model had repeatedly failed, so excluding them favours PURELLM. Counting every bypassed row as a failure gives 18/30, so the unaided pass count lies between 18/30

and 29/30; the baseline has not been rerun with the bypass disabled. The PURELLM figure in Table 4 (29/30; 18/19 on live rows) is the recount over the per-domain files; the noise tables (Appendix J) use the 170344 file, which reports 30/30. The one-test difference is Curie’s law, which fails in the per-domain recount and passes in the 170344 run. Explicit hardcoding audits should be standard in any benchmark involving LLM-based SR.

11.2 M3 vs. M4 Trade-off

With the measurement bug resolved, M3 and M4 are **close to equivalent on R^2** : both reach median $R^2 = 1.000000$ at every tested noisy condition and sample size, though M3’s *recovery rate* dips to 96.7% at $\sigma = 0.1\%$ (Table 9) and at $n = 500/750/1000$ (Table 11), while M4 is 100% throughout. The distinction is: M3 is interpretability-optimised (exact symbolic formulas); M4 is efficiency-optimised ($\sim 1.1\text{--}1.2\times$ faster, near-zero RMSE).

Under the $R^2 > 0.9999$ rule the tie rate is 81.3% on the noise sweep (122/150) and 95.6% on the sample-complexity sweep (172/180), or 89.1% over the 330 pairs combined (Table 12). Direct computation confirms that, on the noise sweep, M4 never loses to M3 on R^2 beyond numerical noise ($|\Delta R^2| \leq 10^{-6}$), and that on the sample-complexity sweep M3 is never higher than M4 under any tie rule.

12 Limitations

1. **Benchmark scope.** 30 equations, 2–5 input variables. Higher-dimensional and PDE targets remain untested.
2. **Memorisation scope.** The hardcoding audit detects verbatim recall but not partial memorisation.
3. **Baseline breadth.** Only M3 and M4 covered by the sweep; adding PySR and DSR sweeps would strengthen generalisation claims.
4. **M3 noiseless recovery.** At $\sigma = 0\%$, M3’s strict recovery rate is 76.7% (23/30, Table 9), not 100%.
5. **No formula complexity metric.** Expression tree depth and operator count should accompany R^2 in future evaluations.
6. **[Resolved, confirmed 2026-09-27.]** The project’s CI dashboard (July 9 run) should not be cited: its own upstream notebooks (NB-01–NB-06) reported “not found” for that run, while its summary asserted “0 open issues” and specific claims resolved. `build_report.py`’s `notebook_execution_status()` (commit 9d7a833f, 2026-07-29, tagged `FIX-DASHBOARD-STALE-GREEN`) now checks per-cell `execution_count` and hard-fails (`sys.exit(1)`) rather than publish a stale green summary over incomplete notebook output; confirmed present and active in the live repository. The July 9 dashboard itself remains uncitable, but the pipeline defect is fixed and cannot recur silently.
7. **[Resolved.]** The legacy `exp2` baseline-lock threshold fix ($R^2 \geq 0.999999$ replacing $R^2 \geq 0.9999$, which had inflated 74/90 to 81/90) is now live-confirmed at 74/90 against

a fresh CI run, and reproduced independently a second time via a patched Gate C schema check; 81/90 should not be cited as current.

13 JMLR Readiness Assessment

Note (v3.0 update, July 2026). The readiness metrics in Table 15 below are based on the v2 benchmark run (30 benchmark equations, noiseless/noise sweep protocol). The main paper (`jmlr_paper_main.tex`) now reports v3.0 results on 74 DeFi tasks under a 40%/60% aggressive extrapolation split (the result file does not record whether the single-axis or PCA-directed variant was used): HypatiaX’s near-perfect-success figures are **90.5 %** (67/74) raw uncorrected / **59.5 %** (44/74) corrected — resolved by cross-reference to the main paper’s Hybrid Decision-Attribution Bug section (`jmlr_paper_main.tex`, § “The Hybrid Decision-Attribution Bug”), which shows that the corrected version does *not* eliminate the 5 Pure-LLM catastrophic failures: the hybrid routes to the LLM on all 5 and its zero-catastrophic report was masked by the same attribution bug (corrected catastrophic count: 5/74, identical to Pure LLM; main paper Table `tab:main_results` recomputed 2026-09-28 from the same file). **Independently re-verified (2026-09-27)** by direct recomputation against the canonical seed-42 file retrieved from `github.com/sednabcn/LLM-HypatiaX-DEV` (md5 `cbf503d0b3985b4c1bccf7ed18e0316b`, canonical `hypatiax/data`; recomputed 2026-09-29); see also Post-Submission Report, § “Item 2.” These are a *different and more stringent* evaluation than the noiseless recovery metrics here; the v2 benchmark readiness assessment remains valid for this document’s scope.

14 Conclusions

1. **Both M3 and M4 achieve 100% recovery** at all tested noisy conditions (0.05–1% noise) and all sample sizes (50–1000).
2. **M4 reaches 100% at $\sigma = 0$** (noiseless, v2-corrected); **M3 reaches only 76.7% (23/30) at $\sigma = 0$** (Table 9). **HDSv50_2** (M6, tools track) trails at 70.0% (21/30) under the separate six-method noiseless protocol (30 equations; Table 4), with nine failing equations (Arrhenius rate constant, Nernst equation, dielectric polarisation, capacitor energy, Coulomb’s law, kinetic energy, Gaussian/normal PDF, Bose–Einstein occupation number, Fermi–Dirac occupation number).
3. **M4 is $\sim 1.1\text{--}1.2\times$ faster** with near-zero RMSE ; the throughput advantage is modest, not decisive.
4. **EHSDeFi sits close to AI Feynman 2.0’s published figure:** 76.7% vs. 79.3% (−2.6 pp) on this work’s own custom 30-equation benchmark. **HDSv50_2 falls further below it:** 70.0% (−9.3 pp). Published baselines were evaluated on all 100 Feynman equations under SRBench, a different equation set entirely; the AI Feynman figure is shown for general context only, not as a head-to-head comparison.
5. **The Newton failure in v1 was a measurement bug:** absolute R^2 threshold misclassified correct formulas for tiny-scale equations. The relative threshold fix is a general contribution to SR benchmarking.

Table 15: JMLR readiness assessment (v2 benchmark results, post bug-fix). See note above for v3.0 DeFi results reported in the main paper.

Criterion	M3	M4	Required action
Accuracy (R^2)	PASS	PASS	Both: median $R^2 = 1.000000$
Noise robustness	COND.	STRONG	M3 76.7% at $\sigma=0$, 96.7–100% at $\sigma > 0$ (Table 9); M4 100% at all σ
Data efficiency	STRONG	STRONG	Convergence at $n = 50$ for both (noiseless only)
Catastrophic failures	CLEAN	CLEAN	0/30 for both; document Newton bug
Computational cost	STRONG	STRONG	Both < 8 s/eq across all σ (Table 10); no evidence of an offline-search spike
Statistical significance	PASS	PASS	Add Wilcoxon test; tie rates recomputed in §9 (81.3% noise, 95.6% SC)
Hardcoding disclosure	REQUIRED		PURELLM: disclose 11/30 hardcoded
Overall verdict	READY	READY	Ready on accuracy/robustness grounds

6. **Four methodological issues corrected:** undisclosed hardcoding, scale-biased RMSE, missing significance tests, mismatched thresholds.

References

- Luca Biggio, Tommaso Bendinelli, Alexander Neitz, Aurelien Lucchi, and Giambattista Parascandolo. Neural symbolic regression that scales. In *Proceedings of the 38th International Conference on Machine Learning (ICML)*, 2021. URL <https://arxiv.org/abs/2106.06427>.
- Subbarao Kambhampati, Karthik Valmeekam, Lin Guan, Kaya Stechly, Mudit Verma, Siddhant Bhambri, Lucas Saldyt, and Anil Murthy. Llms can’t plan, but can help planning in llm-modulo frameworks. *arXiv preprint arXiv:2402.01817*, 2024.
- William La Cava, Patryk Orzechowski, Bogdan Burlacu, Fabrício Olivetti de França, Marco Virgolin, Ying Jin, Michael Kommenda, and Jason H. Moore. Contemporary symbolic regression methods and their relative performance. In *NeurIPS 2021 Datasets and Benchmarks Track*, 2021. URL <https://arxiv.org/abs/2107.14351>.
- Kazem Meidani, Parshin Shojaee, Chandan K. Reddy, and Amir Barati Farimani. SNIP: Bridging mathematical symbolic and numeric realms with unified pre-training. In *The*

Twelfth International Conference on Learning Representations (ICLR), 2024. URL <https://openreview.net/forum?id=KZSEgJGPxu>.

Brenden K. Petersen, Mikel Landajuela Larma, T. Nathan Mundhenk, Claudio P. Santiago, Soo K. Kim, and Joanne T. Kim. Deep symbolic regression: Recovering mathematical expressions from data via risk-seeking policy gradients. In *International Conference on Learning Representations (ICLR)*, 2021. URL <https://arxiv.org/abs/1912.04871>.

Ali Satvaty, Suzan Verberne, and Fatih Turkmen. Undesirable memorization in large language models: A survey. *arXiv preprint arXiv:2410.02650*, 2024. URL <https://arxiv.org/abs/2410.02650>.

Michael Schmidt and Hod Lipson. Distilling free-form natural laws from experimental data. *Science*, 324(5923):81–85, 2009.

Parshin Shojaee, Kazem Meidani, Amir Barati Farimani, and Chandan K. Reddy. Transformer-based planning for symbolic regression. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. URL <https://arxiv.org/abs/2303.06833>.

Silviu-Marian Udrescu and Max Tegmark. Ai feynman: A physics-inspired method for symbolic regression. *Science Advances*, 6(16):eaay2631, 2020.

Silviu-Marian Udrescu, Andrew Tan, Jiahai Feng, Orisvaldo Neto, Tailin Wu, and Max Tegmark. AI Feynman 2.0: Pareto-optimal symbolic regression exploiting graph modularity. *Advances in Neural Information Processing Systems*, 33:4860–4871, 2020. URL <https://proceedings.neurips.cc/paper/2020/hash/33a854e247155d590883b93bca53848a-Abstract.html>.

Appendix A. Reproducibility Figure Generation Inventory

This appendix documents the auxiliary figures produced by `generate_figures.py` for reproducibility purposes. It is distinct from the figures actually embedded in the compiled manuscript (see `figures_inventory.tex`, 5 files, listed in `jmlr_paper_main.tex`).

Table 16: Complete figure inventory; all files in `figures/`. All 13 generated figures must be regenerated from v2 result JSONs.

Filename	Sweep	Section	Content
<code>fig1_r2_vs_noise.pdf</code>	Noise	6	Median R^2 vs σ
<code>fig2_rmse_vs_noise.pdf</code>	Noise	6	Median RMSE vs σ
<code>fig3_time_vs_noise.pdf</code>	Noise	6	Avg time vs σ
<code>fig4_r2_vs_n.pdf</code>	SC	7	Median R^2 vs n
<code>fig5_rmse_vs_n.pdf</code>	SC	7	Median RMSE vs n
<code>fig6_time_vs_n.pdf</code>	SC	7	Median time vs n
<code>fig7_recovery_vs_noise.pdf</code>	Noise	6	Recovery rate vs σ
<code>fig8_recovery_vs_n.pdf</code>	SC	7	Recovery rate vs n
<code>fig9_minr2_vs_noise.pdf</code>	Noise	6	Min R^2 vs σ
<code>fig10_r2_boxplot_noise.pdf</code>	Noise	8	Per-eq R^2 box plots
<code>fig11_recovery_heatmap.pdf</code>	Both	8	Recovery heatmap $\sigma \times n$
<code>fig_runtime_comparison.png</code>	—	5	Runtime bar chart, 6 methods
<code>fig_comparative_table.png</code>	—	5	Domain $\times$ method table

Generation note: Regenerate from `noise_sweep_20260316_192711.json` and `sample_complexity_20260316` (v2 corrected harness; see Section D for v1 vs. v2 provenance).

Relationship to the submitted PDF: the 13 files above are reproducibility documentation only and are not `\includegraphics`d anywhere in this document or in the main manuscript. The compiled manuscript embeds a separate, disjoint set of 5 figures directly via `\includegraphics` (see `jmlr_paper_main.tex`). Only those 5 files are bundled in `submission/sources/figures/`; the 13 listed here should be regenerated from source by anyone who wants them, not shipped in the submission package.

Appendix B. Method Abbreviations

Short	Code	Full name
PURELLM	M1	PureLLM Baseline (core)
IMPNN	M2	ImprovedNN (core)
EHSDeFi	M3	EnhancedHybridSystemDeFi (core)
HLLMNN	M4	HybridSystemLLMNN all-domains (core)
SYMLLM	M5	SymbolicEngineWithLLM (tools)
HDSv50_2	M6	HybridDiscoverySystem v50_2 (tools)

Appendix C. PureLLM Required Disclosure

Required in any JMLR submission including PURELLM results:

The PureLLM Baseline does not call the language model for 11 of 30 benchmark equations (`is_hardcoded: true`): for 8 it returns the benchmark’s own ground-truth formula and for 3 a fixed form whose parameters are fitted to the data by least squares. Its pass rate is 29/30 over all rows and 18/19 on the 19 live rows (where the model was called). The live figure favours PureLLM because the bypass was added for equations on which the model had failed, so the unaided pass count lies between 18/30 and 29/30; the baseline has not been rerun with the bypass disabled.

Appendix D. Benchmark Reproducibility Statement

Code, Data, and Protocol

All experimental results in the main paper’s *Benchmark Design* section were produced using `experiment_protocol_benchmark_v2.py` (version 2.0, 2026-03-02), which will be released alongside the final paper. The protocol file encodes all data-generation functions, random seeds, noise levels, and recovery thresholds as class constants and is self-documenting:

```
python experiment_protocol_benchmark_v2.py --describe
```

The 30-equation benchmark is defined in `_build_feynman.equations()` (lines 480–953 of the protocol file; the function name uses the label “feynman”, but the set is custom, not a Feynman subset). Each equation specifies its identifier, ground-truth formula, variable ranges, and generating function; data are produced analytically, eliminating external dataset dependencies for the physics equations.

Exact Reproduction Commands

Phase 2 (noisy, $R^2 > 0.995$).

```
python run_comparative_suite_benchmark_v2.py \  
  --methods 1 --samples 200 --no-llm-cache
```

Output: `protocol_core_noisy_<timestamp>.json`. The `--no-llm-cache` flag forces independent fresh API calls for all 30 Pure LLM evaluations; this was verified via cache-clearing.

Phase 3 (noiseless, $R^2 \geq 0.999999$, Table 4). The command and output below come from the one verified noiseless source file, `protocol_core_noiseless_20260812.170344.json`, whose internal `script` field is quoted directly.

```
python run_protocol_benchmark_core.py \  
  --noiseless --threshold 0.999999 \  
  --nn-seeds 3 --samples 200 \  
  --method-timeout 900 \  
  --pysr-timeout 1100
```

Script version: `run_protocol_benchmark_core.py v2.2 (sign-fix + log-widen + domain-guard + formula-hash + complexity-score + json-export + extrap_r2_far-fix)`, per the source file’s own `script` field.

Output: `protocol_core_noiseless_20260812.170344.json` (timestamp 2026-08-12T17:03:44; 30 equations across 11 non-Feynman domains — biology, chemistry, electrochemistry, electromagnetism, electrostatics, magnetism, mechanics, optics, probability, quantum, thermodynamics — run against all 6 methods, with per-method pass counts as read from that file, which Table 4 reproduces except for the PURELLM pass count (29/30 in the table, from the per-domain recount; 30/30 in this file); the other counts in this file are SYMLLM 28/30, HLLMNN 30/30, EHSDEFI 23/30, HDSv50.2 21/30, IMPNN 0/30; wall-clock and hardware not recorded in the source file, so no timing claim is made here.

Note on timeout upgrade: `--method-timeout` was set to 300s for tests 1–18 and upgraded to 900s for tests 19–30. Arrhenius (test 4) ran under the 300s limit and the harness recorded a 27574s wall-clock before the run ended; this is recorded as a genuine failure independent of the v2 bug fix. **Revised mechanism (Phase 3 audit, item 10b):** PySR is invoked via `juliacall`, which embeds Julia in-process rather than spawning a separate OS process, so there was no external Julia PID being “killed.” The 27574s figure instead reflects a gap in the harness’s own timeout wiring: the outer `ThreadPoolExecutor` timeout fires at 300s but abandons the background thread without a reference back to the underlying call, which then continues to run unobserved.

Nernst (test 6) also timed out under the 300s default; counted as failure.

Random Seeds

- Base seed: 42 (class constant `BenchmarkProtocol.seed = 42`, line 1091 of the protocol file). Matches the Core 15 benchmark seed for cross-campaign consistency.
- Per-equation seed: `seed + offset × 7`, where `offset` is the 0-based index in the equation list (line 1208: `eq.generate(n, noise, seed=s + offset * 7)`). This ensures each of the 30 equations receives a distinct, reproducible seed regardless of call order.
- Neural network seeds (Phase 3): 3 independent seeds per equation; median R^2 reported. Exact seed values are logged in the output JSON.

Software Environment

Table 17: Software and hardware environment for benchmark runs (verified 18 March 2026 via `hardware_info.py` and `pip show`).

Component	Version / details
Python	3.12.2
PySR	1.5.9
Julia runtime	1.12.2
juliacall	0.9.26 (Python $\leftrightarrow$ Julia bridge used by PySR)
NumPy	2.3.5
SciPy	1.16.3
SymPy	1.14.0
Pandas	2.3.3
Matplotlib	3.10.7
Seaborn	0.13.2
scikit-learn	1.7.2
Anthropic SDK	0.73.0 (claude-sonnet-5, temperature 0.25)
Hardware	Intel Celeron T3100 @ 1.90 GHz (2 cores); 3 GB DDR2; no GPU
OS	Kali GNU/Linux Rolling
PySR populations	2 (hardware limit: 2-core CPU)

Note on Anthropic SDK version compatibility. `anthropic==0.73.0` is substantially newer than early-2024 releases. The Messages API interface is stable across 0.18+ versions; earlier installations may require minor call-site adjustments but the prompts and response parsing are otherwise unchanged.

PySR uses Julia under the hood; the 1100s `--pysr-timeout` (`FEYNMAN_TIMEOUT`, the inner PySR fit budget—distinct from the 900s outer `--method-timeout` above, which governs the per-method wall-clock allowance, not the PySR fit itself) was required to accommodate Julia JIT initialisation on the first run. Subsequent equations in the same process are faster. The 27574s Arrhenius figure is a harness-timeout artifact, not evidence that PySR’s complexity search fails to converge on single-variable flat-landscape equations; see the revised mechanism note above.

Measurement Correction ($v1 \rightarrow v2$)

A bug was identified on 2026-03-16 in `evaluate_llm_formula` and corrected before final submission. The function used a hardcoded absolute sum-of-squares guard (`ss_tot > 1e-10`) that silently returned $R^2 = 0$ for equations whose output magnitudes are far below unity (Newton gravity $\sim 10^{-11}$ N, Zeeman splittings $\sim 10^{-23}$ J, Coulomb forces at μC scale). The fix scales the threshold relative to the data:

```
# v1 (buggy):
r2 = 1 - (ss_res / ss_tot) if ss_tot > 1e-10 else 0

# v2 (fixed):
```

```
_tol = 1e-10 * float(np.max(np.abs(y_true))**2) * len(y_true)
r2 = 1 - (ss_res / ss_tot) if ss_tot > max(_tol, 1e-300) else 0
```

The same fix was applied at three locations:

1. `evaluate_llm_formula` (line 462) — primary failure site
2. `train_nn` (line 384) — NN self-evaluation
3. Ensemble-blend R^2 calculation (line 700) — candidate scoring

Additional fixes in the same commit:

- `force_llm` flag: `hybrid_predict` now correctly bypasses NN training when the LLM formula passes the threshold.
- PureLLMBaseline passthrough: `generate_llm_formula` now validates non-empty `python_code` before accepting.
- Training epochs: corrected from 300 to 1000 to satisfy the learning rate scheduler’s warm-restart requirement (≥ 1000 epochs).
- Log-space guard: `_is_pole` heuristic now unconditionally enables log-space training when `y_range > 3.0` decades, fixing suppression on wide-range power-law equations.

The *Summary of Changes* table in Supplementary A (*Routing Improvements for Hybrid LLM-Neural-Network Systems*), and Section D of this document, summarise all aggregate recovery changes. All results reported in the paper use the v2 corrected harness. The v1 JSON output files are retained for audit purposes and are available upon request.

Per-Equation v2 Data

The complete per-equation R^2 values under the v2 corrected harness are stored in:

- `protocol_core_noiseless_20260304_154510.json` — Phase 3 noiseless, all 6 methods, 30 equations.
- `noise_sweep_20260316_192711.json` — M3/M4 noise sweep ($\sigma \in \{0, 0.05, 0.1, 0.5, 1\}\%$, v2 corrected).
- `sample_complexity_20260316_193447.json` — M3/M4 sample complexity sweep ($n \in \{50, 100, 200, 500, 750, 1000\}$, v2 corrected).

These files will be released as supplementary material. The v1 superseded files (`noise_sweep_20260315_091018.json`, `sample_complexity_20260315_124310.json`) should not be used for the final submission figures.

PureLLM Disclosure

Following Satvaty et al. (2024), each PureLLM noiseless row was audited for whether the language model was called. **11 of 30 PureLLM rows bypass the model (is_hardcoded=True):** Michaelis–Menten, Logistic growth, Allometric scaling, Nernst, Clausius–Mossotti, Lorentz force, Gaussian PDF, Zeeman energy, Bose–Einstein, Planck radiance, and Fourier heat. Eight of these return the benchmark’s ground-truth formula; Michaelis–Menten, Logistic growth and Allometric scaling use a fixed form with least-squares parameters fitted to the data. PureLLM passes 29/30 over all rows and 18/19 on the 19 live rows (where the model was called); because the bypass was added for equations on which the model had failed, the live figure favours PureLLM and the unaided pass count lies between 18/30 and 29/30.

PureLLM is listed in all tables, and the main paper reports its comparisons with and without the bypassed rows.

A controlled re-run with `--no-llm-cache` confirmed that all 30 equations issued independent API calls and produced identical scores (median $R^2 = 0.9976$, std = 2.5×10^{-4}), ruling out any caching artefact.

Citation Key Corrections

The following citation key conflicts have been resolved in `references.bib` (updated 2026-06):

1. `cranmer2023interpretable` renamed to `cranmer2023pysr` (canonical key matching all `\cite` calls in the main paper; same paper arXiv:2305.01582). **Resolved.**
2. `udrescu2020aifeynman` duplicate removed; `udrescu2020ai` retained as the single canonical key in `references.bib`. This supplementary document uses `udrescu2020aifeynman` in its own inline `thebibliography` and remains self-consistent. **Resolved.**
3. `meidani2023snip` (arXiv preprint key) replaced by `meidani2024snip` (published ICLR 2024 key) in `references.bib`. **Resolved.**
4. `lacava2021contemporary` entry confirmed present in `references.bib`. **No action needed.**

SNIP Comparison Note

SNIP (Meidani et al., 2024) does *not* report recovery rates. It evaluates symbolic regression using a Pareto front of median R^2 vs. equation complexity on 119 Feynman equations, reporting median $R^2 = 0.882$ (vs. AI Feynman baseline: 0.798). Placing SNIP’s median R^2 alongside HypatiaX’s recovery rate (% at $R^2 > 0.9999$) would be a category error. The only valid comparison uses the shared median R^2 axis: HypatiaX Hybrid DeFi achieves median $R^2 = 1.0000$ on our 30-equation subset vs. SNIP’s 0.882 on the full 119-equation SRBench set (different subsets; comparison is indicative only). The Nguyen-12 benchmark table in the main paper provides this comparison.

Appendix E. Hyperparameter Settings

Table 18: PySR Configuration for Symbolic Regression

Parameter	Value
Population size	100
Generations	500
Populations (parallel)	2 (hardware limit: 2-core CPU)
Crossover probability	0.8
Mutation probability	0.1
Complexity penalty	0.01
Tournament size	3
Elitism	5
Max expression depth	15
Optimization iterations	50
Validation Thresholds	
Acceptance R^2 (Criterion 1)	0.99
Validation score (Criterion 1)	30
Acceptance R^2 (Criterion 2)	0.95
Validation score (Criterion 2)	80
Extrapolation R^2	0.80
Numerical stability (α)	0.95

Table 19: Complete Hyperparameter Specifications for All Methods

Method	Parameter	Value
Neural Network	Architecture	[64, 32, 1]
	Activation	ReLU
	Optimizer	Adam
	Learning rate	0.01
	Batch size	Full batch (160 samples)
	Epochs	200
	Weight initialization	Xavier uniform
	Dropout	None
Pure LLM	Model	claude-sonnet-5 [corrected: was claude-sonnet-4-20250514]
	Temperature	0.25 [corrected: was 0.0 (deterministic)]
	Max tokens	1024
	Top-p	1.0
	Few-shot examples	2-3 per domain
	Prompt template	Domain-specific
	Retry on failure	3 attempts
Hybrid v50_2	SR iterations	50
	SR populations	2 (hardware limit: 2-core CPU)
	Binary operators	[+, -, *, /]
	Unary operators	[exp, log, sqrt, sin, cos]
	Parsimony coefficient	0.01
	Tournament size	10
	Mutation rate	0.1
	Crossover rate	0.9
	LLM candidates	5
	LLM temperature	0.25 [corrected: was 0.3]
	Max complexity	30
	Validation threshold (R^2)	0.99

Appendix F. Detailed Experimental Results

F.1 Pure LLM Baseline: Per-Case Results

Table 20: Pure LLM performance by test case (pinned DeFi benchmark, seed 42). ---: the result file holds no numeric R^2 for that case.

Case	R^2	Time (s)
Value at Risk at 95%	1.0000	8.23
Value at Risk at 99%	1.0000	6.84
Partial liquidation amount	1.0000	12.01
Constant product formula	1.0000	6.71
Simple Staking APY	1.0000	5.95
Loan-to-Value	-1.41×10^5	7.61
Spot price from AMM	1.0000	6.79
LP share percentage	-1.8739	5.94
Long position unrealized PnL	1.0000	4.97

Case	R^2	Time (s)
Short position unrealized PnL	1.0000	4.74
Funding rate cost	1.0000	7.26
Validator commission adjusted	1.0000	5.69
Slashing penalty	—	14.26
Protocol reserve accumulation	1.0000	7.09
Leveraged position notional	1.0000	6.84
Cross-margin available balance	1.0000	8.64
Realized PnL for long	1.0000	4.88
LP fee earnings	1.0000	8.81
Multi-day Value at Risk	1.0000	7.02
ES scaling for multi-day	1.0000	7.66
Annualised Portfolio tracking error	1.0000	7.26
Incremental VaR	1.0000	8.15
Reserve ratio	1.0000	6.63
Impermanent loss breakeven fee rate	1.0000	11.84
Liquidation Price Long	—	7.56
Liquidation Price Short	—	6.56
Constant Product Price Impact	1.0000	7.78
Effective Leverage	-12.4502	14.12
Borrowing Interest	1.0000	6.67
Compounding Staking Returns	1.0000	7.21
Portfolio Sharpe Ratio	1.0000	5.50
APY calculation	1.0000	8.62
Capital efficiency	—	8.40
AMM output amount	1.0000	10.10
Price slippage percentage	-3.2163	6.97
Information ratio	1.0000	5.33
Utilization rate of DeFi	—	6.64
Borrow APY from utilization	1.0000	6.45
Supply APY from borrow	1.0000	6.38
Health factor	1.0000	9.74
Impermanent loss percentage	-2.1798	7.72
Options Delta	—	16.66
Collateral ratio	—	7.55
Funding rate cost (extended)	1.0000	7.88
Staking reward for fixed lock-up	-0.9246	7.13
Expected Shortfall at 95%	1.0000	7.17
Expected Shortfall at 99%	1.0000	7.20
Optimal LP Position (Kelly)	—	9.26
Concentrated liquidity position width	-2.59×10^8	11.11
Portfolio VaR for two correlated	1.0000	9.19
Position margin ratio	1.0000	6.19
Concentrated liquidity position width (v2)	1.0000	7.66
Spot price from AMM reserve	0.9810	10.03
Black-Scholes Call Price	—	21.86
Black-Scholes Put Price	—	16.19

Case	R^2	Time (s)
Component ES	-0.2749	9.61
Gamma of option	—	12.63
Vega of option	—	9.73
Multi-Collateral LTV	1.0000	8.55
Correlated Portfolio VaR	1.0000	8.41
Impermanent loss in constant product	1.0000	9.08
Constant product formula (multivariate)	1.0000	5.91
Convexity Adjustment	0.9998	18.49
Liquidation price for leveraged long	—	10.06
Liquidation price for leveraged short	—	6.68
Maximum safe leverage	—	7.92
Required collateral	—	8.04
Forward price for derivative	1.0000	6.74
Portfolio Expected Shortfall for correlated	—	4.99
Call option intrinsic	-3.5798	4.68
Put-call parity	1.0000	7.33
Simple options moneyness	-50.5674	4.92
Uniswap V3 virtual	-9.41×10^6	14.65
Theta of option	—	21.12

F.2 DeFi-Optimized HypatiaX: Per-Case Results

Table 21: HypatiaX performance by test case (pinned DeFi benchmark, seed 42). R^2 is that of the sub-method named by the routing decision, not hybrid’s self-reported value; ---: that sub-method returned no numeric R^2 . Decisions without an independent baseline (ensemble, v4_residual_nn) show hybrid’s own value.

Case	R^2	Time (s)	Decision
Value at Risk at 95%	1.0000	1.34	11m
Value at Risk at 99%	1.0000	1.08	11m
Partial liquidation amount	1.0000	1.13	11m
Constant product formula	1.0000	1.30	11m
Simple Staking APY	1.0000	1.30	11m
Loan-to-Value	-1.41×10^5	1.27	11m
Spot price from AMM	1.0000	1.07	11m
LP share percentage	-1.8739	1.11	11m
Long position unrealized PnL	1.0000	1.26	11m
Short position unrealized PnL	1.0000	1.78	11m
Funding rate cost	1.0000	1.12	11m
Validator commission adjusted	1.0000	1.22	11m
Slashing penalty	—	1.46	11m

Case	R^2	Time (s)	Decision
Protocol reserve accumulation	1.0000	1.29	11m
Leveraged position notional	1.0000	1.05	11m
Cross-margin available balance	1.0000	1.20	11m
Realized PnL for long	1.0000	1.41	11m
LP fee earnings	1.0000	1.30	11m
Multi-day Value at Risk	1.0000	1.33	11m
ES scaling for multi-day	1.0000	1.25	11m
Annualised Portfolio tracking error	1.0000	1.21	11m
Incremental VaR	1.0000	2.09	11m
Reserve ratio	1.0000	1.35	11m
Impermanent loss breakeven fee rate	1.0000	1.59	11m
Liquidation Price Long	—	1.18	11m
Liquidation Price Short	—	1.28	11m
Constant Product Price Impact	1.0000	1.78	11m
Effective Leverage	-12.4502	1.87	11m
Borrowing Interest	1.0000	1.26	11m
Compounding Staking Returns	1.0000	1.26	11m
Portfolio Sharpe Ratio	1.0000	1.13	11m
APY calculation	1.0000	1.10	11m
Capital efficiency	—	1.68	11m
AMM output amount	1.0000	1.16	11m
Price slippage percentage	-3.2163	1.27	11m
Information ratio	1.0000	1.18	11m
Utilization rate of DeFi	—	1.69	11m
Borrow APY from utilization	1.0000	1.27	11m
Supply APY from borrow	1.0000	1.17	11m
Health factor	1.0000	1.57	11m
Impermanent loss percentage	-2.1798	1.47	11m
Options Delta	—	1.76	11m
Collateral ratio	—	2.30	11m
Funding rate cost (extended)	1.0000	1.29	11m
Staking reward for fixed lock-up	-0.9246	1.39	11m
Expected Shortfall at 95%	1.0000	1.15	11m
Expected Shortfall at 99%	1.0000	1.38	11m
Optimal LP Position (Kelly)	0.0000	2.59	nn_fallback
Concentrated liquidity position width	-2.59×10^8	1.39	11m
Portfolio VaR for two correlated	1.0000	1.66	11m
Position margin ratio	1.0000	1.38	11m
Concentrated liquidity position width (v2)	1.0000	1.20	11m
Spot price from AMM reserve	0.9810	1.33	11m
Black-Scholes Call Price	-1.8315	4.21	nn
Black-Scholes Put Price	-0.6802	3.36	nn
Component ES	-0.2749	1.16	11m
Gamma of option	-0.1060	4.37	nn
Vega of option	0.6285	4.61	nn

Case	R^2	Time (s)	Decision
Multi-Collateral LTV	1.0000	1.39	11m
Correlated Portfolio VaR	1.0000	1.44	11m
Impermanent loss in constant product	1.0000	1.26	11m
Constant product formula (multivariate)	1.0000	1.29	11m
Convexity Adjustment	0.9998	1.39	11m
Liquidation price for leveraged long	—	1.32	11m
Liquidation price for leveraged short	—	1.64	11m
Maximum safe leverage	—	1.34	11m
Required collateral	—	1.08	11m
Forward price for derivative	1.0000	1.92	11m
Portfolio Expected Shortfall for correlated	—	1.34	11m
Call option intrinsic	-3.5798	1.29	11m
Put-call parity	1.0000	1.36	11m
Simple options moneyness	-50.5674	1.38	11m
Uniswap V3 virtual	-9.41×10^6	1.24	11m
Theta of option	-0.8034	13.51	nn

F.3 Complete Test Suite Details

Table 22: Ground Truth Formulas with Exact Constants

Equation	Formula	Constants
Arrhenius	$k = A \exp(-E_a/(RT))$	$A = 10^{11}$, $E_a = 80000$ J/mol, $R = 8.314$ J/(mol·K)
Henderson-Hasselbalch	$\text{pH} = \text{p}K_a + \log_{10}([A^-]/[\text{HA}])$	$\text{p}K_a = 6.5$
Rate Law	$r = k[A]^2[B]$	$k = 0.5$ M ⁻² s ⁻¹
Allometric Scaling	$Y = aM^b$	$a = 3.5$, $b = 0.75$
Michaelis-Menten	$v = V_{\max}[S]/(K_m + [S])$	$V_{\max} = 50$ μM/s, $K_m = 10$ μM
Logistic Growth	$dN/dt = rN(1 - N/K)$	$r = 0.3$ day ⁻¹ , $K = 1000$
Kinetic Energy	$E = \frac{1}{2}mv^2$	Dimensionless
Gravitational Force	$F = Gm_1m_2/r^2$	$G = 6.674 \times 10^{-11}$ N·m ² /kg ²
Ideal Gas Law	$P = nRT/V$	$R = 8.314$ J/(mol·K)
Impermanent Loss	$\text{IL} = 2\sqrt{r}/(1 + r) - 1$	Dimensionless
Price Impact	$\Delta p = \Delta x/(x + \Delta x)$	Dimensionless
Constant Product	$y = k/x$	$k = 10^6$
VaR 95%	$\text{VaR} = P\sigma \cdot z_{0.95}$	$z_{0.95} = 1.645$
Liquidation Price	$p_{\text{liq}} = p_0(1 - 1/(L \cdot m))$	$m = 0.8$ (maintenance margin)
Portfolio Variance	$\sigma_p = \sqrt{\sigma_1^2 + \sigma_2^2 + 2\rho\sigma_1\sigma_2}$	Dimensionless

Appendix G. Statistical Test Details

Experiment 1 (exp1_five, Core-15)

G.1 Mann-Whitney U Test for Extrapolation Performance

For medium extrapolation regime ($2\times$ training range):

Test Setup:

- **Sample sizes:** $n_{\text{Hybrid v50.2}} = 12$, $n_{\text{Neural Network}} = 15$
- **Null hypothesis:** $H_0 : E_{\text{Hybrid v50.2}} \geq E_{\text{Neural Network}}$ (Hybrid v50.2 is not better)
- **Alternative hypothesis:** $H_1 : E_{\text{Hybrid v50.2}} < E_{\text{Neural Network}}$ (Hybrid v50.2 is better)
- **Significance level:** $\alpha = 0.05$ (one-tailed test)

Descriptive Statistics:

- **Hybrid v50.2:** Mean = 1.362e+26%, Median = 2415.2%, SD = 4.717e+26%, Range = [1.3, 1.634e+27]
- **Neural Network:** Mean = 8899923.4%, Median = 13283.9%, SD = 33577002.8%, Range = [216.4, 130261180.6]

Test Results:

- **Test statistic:** $U = 74.0$
- **P-value:** $p = 2.25e - 01$ (one-tailed, normal approximation with tie correction – see `_render_stat_test.tex()` in `generate_tables.py`)
- **Conclusion:** $p \geq 0.05 \Rightarrow$ Fail to reject H_0 at $\alpha = 0.05$

G.2 Effect Size Calculation (Cohen's d)

$$\mu_{\text{Neural Network}} = 8899923.4\%, \quad \sigma_{\text{Neural Network}} = 33577002.8\% \quad (1)$$

$$\mu_{\text{Hybrid v50.2}} = 1.362e + 26\%, \quad \sigma_{\text{Hybrid v50.2}} = 4.717e + 26\% \quad (2)$$

$$s_{\text{pooled}} = \sqrt{\frac{\sigma_{\text{Neural Network}}^2 + \sigma_{\text{Hybrid v50.2}}^2}{2}} = 3.336e + 26\% \quad (3)$$

$$d = \frac{8899923.4 - 1.362e + 26}{3.336e + 26} = -0.41 \quad (4)$$

Conservative Estimate: Using only Neural Network standard deviation (most conservative approach):

$$d_{\text{conservative}} = \frac{8899923.4 - 1.362e + 26}{33577002.8} = -4.056e+18 \quad (\text{interpret magnitude per Cohen 1988 conventions}) \quad (5)$$

Alternative Calculation (Glass's Δ): Using only the control group (Neural Network) standard deviation:

$$\Delta = \frac{8899923.4 - 1.362e + 26}{33577002.8} = -4.056e + 18 \quad (6)$$

Interpretation caveat. The Hybrid v50_2 mean and SD are dominated by a single equation whose extrapolation error is $1.634 \times 10^{27} \%$, so the mean-based effect sizes above (including the -4.056×10^{18} “conservative” value, which is $(\mu_{NN} - \mu_H)/\sigma_{NN}$) are not interpretable. The medians (2415.2% against 13283.9%) and the rank-based U test are the meaningful quantities; the test does not reject H_0 . (Recomputed 2026-09-29 from the canonical `hypati`ax/data files; the previous `data_old`-based block gave $n = 11$, $U = 68.0$, $p = 0.234$.)

Experiment 2 (exp2_five, 10-domain suite)

No statistical test is reported for exp2_five: the generator requires both Hybrid v50_2 and Neural Network to have at least one successful, finite extrapolation-error measurement each, and exp2_five currently has zero for both (see the corresponding tables in the “Five-System Comparative Analysis” section of the main paper).

Appendix H. Discovered Expression Examples

H.1 Perfect Recoveries

H.1.1 KINETIC ENERGY

Example 1 (Kinetic Energy Discovery) *Ground truth:* $E = \frac{1}{2}mv^2$

HypatiaX v50_2 discovered:

$$E = ((v \times m) \times v) \times 0.5 = 0.5mv^2 \quad (7)$$

Analysis: *Exact recovery of ground truth. Discovered expression is algebraically identical.*

Interpolation: $R^2 = 1.0000$ on training data ($v \in [1, 10]$ m/s)

Extrapolation: 0.0% error at $v = 50$ m/s ($5\times$ training max)

Physical interpretation: *Correctly captures quadratic dependence on velocity and linear dependence on mass.*

H.1.2 IDEAL GAS LAW

Example 2 (Ideal Gas Law Discovery) *Ground truth:* $P = nRT/V$, where $R = 8.314 \text{ J}/(\text{mol}\cdot\text{K})$

HypatiaX v50_2 discovered:

$$P = n \times (T \times (8.314/V)) \quad (8)$$

Analysis: *Exact recovery including the universal gas constant $R = 8.314$ to full precision.*

Interpolation: $R^2 = 1.0000$ on training data

Extrapolation: 0.0% error across all pressure/temperature/volume ranges tested (up to $5\times$ training range)

Physical interpretation: *On this test case, symbolic regression recovered a constant consistent with the gravitational constant from data alone, without prior knowledge of its value (La Cava et al., 2021).*

H.2 Near-Perfect Recoveries

The Michaelis-Menten and Impermanent Loss equations were recovered with constant error $< 10^{-5}$, achieving $R^2 = 1.0000$ and near-zero extrapolation error (see the discovered-expression examples above).

H.3 Failure Case Analysis

H.3.1 GRAVITATIONAL FORCE (BOTH METHODS FAILED)

Example 3 (Gravitational Force Failure) *Ground truth:* $F = Gm_1m_2/r^2$, where $G = 6.674 \times 10^{-11} \text{ N}\cdot\text{m}^2/\text{kg}^2$

Neural Network Result:

- **Interpolation:** $R^2 = 0.21$ (poor fit; relative error $> 100\%$, meeting the threshold in the main paper’s Interpolation–Extrapolation Decoupling definition)

- **Extrapolation:** Not attempted due to interpolation failure
- **Issue:** Gradient vanishing due to 10^{-11} scale
- **Attempted solution:** Log-transform $\rightarrow$ marginal improvement ($R^2 = 0.35$)
- **Root cause:** Optimization landscapes too flat at this scale for gradient descent

HypatiaX v50_2 Result:

- **Interpolation:** $R^2 = -0.03$ (worse than baseline)
- **Extrapolation:** Not attempted due to interpolation failure
- **Discovered form:** $F = 1.23 \times 10^{13} \frac{m_1 m_2}{r^2}$ (correct functional form, wrong constant by $2 \times 10^{23} \times$)
- **Issue:** PySR constant search range $[10^{-3}, 10^3]$ missed $G \sim 10^{-11}$
- **Attempted solution:** Extend range to $[10^{-15}, 10^{15}] \rightarrow$ no convergence in 50 iterations
- **Root cause:** Search space too large for evolutionary algorithm to explore efficiently

Recommended Solution:

1. **Dimensional analysis preprocessing:** Identify required constant scales before symbolic regression
2. **Logarithmic transformation:** Convert $F = Gm_1m_2/r^2$ to $\log F = \log G + \log m_1 + \log m_2 - 2 \log r$, making the problem linear in log-space
3. **Adaptive constant ranges:** Dynamically adjust search based on data magnitude
4. **Multi-scale normalization:** Normalize inputs/outputs to $[0.1, 10]$ range before discovery, then rescale

Lesson learned: Very small constants ($< 10^{-6}$) require special handling in both neural and symbolic methods. This represents 1/15 test cases (6.7% failure rate) and is a known limitation addressed in Section 12.

H.3.2 HENDERSON-HASSELBALCH (PURE LLM FAILURE)

Example 4 (Henderson-Hasselbalch pH Calculation) Ground truth: $pH = pKa + \log_{10} \left(\frac{[A^-]}{[HA]} \right)$

Pure LLM Generated:

$$pH = pKa \times \frac{[A^-]}{[HA]} \quad (9)$$

Failure Analysis:

- **R^2 score:** -12.3 (predictions anticorrelated with ground truth)

- **Errors:**

1. *Multiplication instead of addition*
2. *Missing logarithm entirely*
3. *Plausible but fundamentally wrong*

- **Root cause:** *LLM training data likely contains more qualitative discussions of pH ("pH increases when conjugate base increases") than quantitative formulas*

- **Pattern matching failure:** *LLM defaulted to simpler multiplicative relationship rather than logarithmic form*

Symbolic Rescue: *PySR correctly discovered logarithmic relationship:*

$$pH = pKa + \log_{10} \left(\frac{[A^-]}{[HA]} \right) \quad (10)$$

Achieving $R^2 = 0.998$, demonstrating the value of hybrid architecture.

Lesson learned: *LLM performance declined on this specialised equation, which is consistent with the main paper’s LLM Reproductive Behaviour finding—generation probability is theoretically predicted to be lower for expressions that are structurally distant from the training corpus (Kambhampati et al., 2024).*

H.4 Complete Discovered Expressions Code Listing

```

1 # Kinetic Energy (PERFECT)
2 discovered: ((v * m) * v) * 0.5
3 ground_truth: 0.5 * m * v**2
4 R2: 1.0000, extrapolation_error: 0.0%
5
6 # Rate Law (PERFECT)
7 discovered: ((A_conc * 0.5) * A_conc) * B_conc
8 ground_truth: 0.5 * A_conc**2 * B_conc
9 R2: 1.0000, extrapolation_error: 0.0%
10
11 # Ideal Gas Law (PERFECT)
12 discovered: n * (T * (8.314 / V))
13 ground_truth: n * 8.314 * T / V
14 R2: 1.0000, extrapolation_error: 0.0%
15
16 # Logistic Growth (PERFECT - algebraically equivalent)
17 discovered: ((N * -0.0003) + 0.3) * N
18 ground_truth: 0.3 * N * (1 - N/1000)
19 expanded: 0.3*N - 0.0003*N^2 (identical)
20 R2: 1.0000, extrapolation_error: 0.0%
21
22 # Price Impact (PERFECT)
23 discovered: swap / (swap + reserve)
24 ground_truth: dx / (x + dx)
25 R2: 1.0000, extrapolation_error: 0.0%
```

```

26
27 # Michaelis-Menten (NEAR-PERFECT)
28 discovered: (S / (S + 10.000007)) * 50.000008
29 ground_truth: 50 * S / (10 + S)
30 constant_error: <1e-5
31 R2: 1.0000, extrapolation_error: 0.0%
32
33 # Impermanent Loss (NEAR-PERFECT)
34 discovered: (2 * sqrt(r)) / (1 + r) - 1.000023
35 ground_truth: 2 * sqrt(r) / (1 + r) - 1
36 constant_error: 2.3e-5
37 R2: 1.0000, extrapolation_error: 0.0%

```

Listing 1: Discovered Symbolic Expressions (7 Perfect Recoveries)

Appendix I. Supplementary Materials

The following supplementary materials are available:

- **S1: Pure LLM Baseline Results** (`S1_pure_llm_results.csv`) — Complete results from Pure LLM baseline experiments.
- **S2: Extrapolation Test Data** (`S2_extrapolation_data.csv`) — Training R^2 scores and extrapolation errors at $1.2\times$, $2\times$, and $5\times$ training ranges.
- **S3: Validation Logs** (`S3_validation_logs.json`) — Complete validation reports for all 131 tests showing the four-layer validation framework in action. Each entry includes dimensional analysis, physics constraints, numerical stability checks, prediction accuracy metrics, timing data, and failure mode classification. Includes validation layer rejection rates (12%, 18%, 23%, 47%).
- **S4: Discovered Symbolic Expressions** (`S4_discovered_expressions.{txt,json}`) — All 131 discovered symbolic expressions in SymPy-compatible format, organized by domain (chemistry, biology, physics, DeFi, finance) with complete metadata including ground truth equations, performance metrics (R^2 , timing, iterations), and algebraic equivalence verification.
- **S5: Timing Breakdown** (`S5_timing_breakdown.csv`) — Detailed computational cost analysis showing component-level timing: LLM initialization (0-39%), symbolic search (50-91%), and validation (7-11%). Demonstrates the $4.3\times$ speedup from LLM-guided initialization versus pure symbolic methods.
- **S6: Domain-Specific Analysis** (`S6_domain_analysis.xlsx`) — Statistical analysis by scientific domain showing success rates: Biology 100%, Chemistry 100%, Physics 100%, DeFi 100%, Finance 77.8%. Includes per-equation breakdowns and domain-specific constraint validation results.
- **S7: Failure Mode Taxonomy** (`S7_failure_mode_taxonomy.pdf`) — Comprehensive taxonomy of 23 distinct failure modes categorized by method (Pure LLM: 8

modes, Neural Network: 3 modes, Symbolic: 5 modes, Hybrid: 2 modes, Integration: 5 modes), severity (8 Critical, 4 High, 9 Medium, 2 Low), and detection layer. Includes detailed examples of silent semantic errors, large-scale extrapolation errors, distributional reasoning failures, and API misuse.

- **S8: Figure Source Data** (`S8_figures_source_data/`) — Source data files for all figures in the paper:

- `figure1_arrhenius_extrapolation.csv` — Arrhenius equation extrapolation comparison
- `figure2_domain_comparison.csv` — Success rates by scientific domain
- `figure3_validation_breakdown.csv` — Four-layer validation effectiveness
- `figure4b_domain_breakdown.csv` — Per-equation performance details
- `figure5_method_comparison.csv` — Overall method comparison
- `figure_5systems_comparison.csv` — Extrapolation data for the five-system comparison; see the “Five-System Comparative Analysis” section of the main paper for the regenerated tables (see Issue 4 correction note above)
- `figure_benchmark_comparison.csv` — Core 15 benchmark results
- `figure_timing_comparison.csv` — Speed-accuracy tradeoff analysis

All supplementary materials are provided in both synthetic versions (matching reported statistics exactly) and real experimental versions (direct from JSON logs) for maximum reproducibility and transparency. Complete documentation and usage examples are provided in accompanying README files.

I.1 Supplementary Data Files

Post-processed results, tables and the paper sources are available in the post-processing and pre-submission repository:

<https://github.com/sednabcn/LLM-HypatiaX-DEV>

The raw benchmark runs are produced and archived in the reproduction repository, <https://github.com/sednabcn/LLM-HypatiaX-REPRO>.

Table 23: Supplementary Data Files

File	Contents
S1_pure_llm_results.csv	Complete Pure LLM test results (n=40) with formulas, R^2 scores, failure modes
S2_extrapolation_data.csv	Extrapolation test data ($n = 27$) at $1.2\times$, $2\times$, $5\times$ training ranges
S3_validation_logs.json	Full validation reports for all 131 tests
S4_discovered_expressions.txt	All discovered symbolic expressions in SymPy format
S5_timing_breakdown.csv	Detailed timing data by component and method
S6_domain_analysis.xlsx	Per-domain performance with statistical tests
S7_failure_taxonomy.pdf	Complete taxonomy of 23 failure modes with examples
S8_figures_source_data/	Raw data for all 8 figures in CSV format

I.2 Reproducibility Materials

Complete step-by-step tutorials for reproducing all experiments are available at: <https://sednabcn.github.io/ai-llm-blog/tutorials/hypatiaux/>

- Tutorial 1: Environment Setup and First Discovery (15 min)
- Tutorial 2: Running Benchmark Experiments (45 min active + 3–8 hours compute)
- Tutorial 3: Statistical Analysis and Publication Figures (30 min)
- Tutorial 4: Custom Applications and Extensions (45 min)

All source code, experimental protocols and data generation scripts are available at: <https://github.com/sednabcn/LLM-HypatiaX-REPRO> (reproduction repository: all tests and benchmarks are run from there; <https://github.com/sednabcn/LLM-HypatiaX-DEV> holds post-processing and pre-submission material only).

I.3 Reproducibility Artifacts

Docker Images:

- `hypatiaux:v1.0` - Full environment with all dependencies
- `hypatiaux:minimal` - Core symbolic regression only (no LLM)
- `hypatiaux:gpu` - GPU-accelerated version for neural network comparison

Source Code: All Python scripts and experimental protocols are available at:

<https://github.com/sednabcn/LLM-HypatiaX-REPRO/tree/master/hypatiaux>

Jupyter Notebooks: The notebooks that do exist are the six paper-integrity audit notebooks NB-01 to NB-06 (citation, cross-reference, section numbering, numerical consistency, figure dependency, and code-quality checks), available at:

<https://github.com/sednabcn/LLM-HypatiaX-DEV/tree/master/notebooks>

Step-by-Step Tutorials:

- Tutorial 1: Environment Setup and First Discovery (15 min)
- Tutorial 2: Running Benchmark Experiments (45 min active + 3–8 hours compute)
- Tutorial 3: Statistical Analysis and Publication Figures (30 min)
- Tutorial 4: Custom Applications and Extensions (45 min)

Available at: <https://sednabcn.github.io/ai-llm-blog/tutorials/hypatiax/>

Appendix J. Noise Sensitivity Analysis

This section reports solve counts for the five noise levels present in the repository’s `protocol_core` result files ($\sigma \leq 1\%$). No results are available for higher noise levels, so no claim is made about a noise threshold.

J.1 Protocol

Five `protocol_core` runs (one noiseless, four noisy; 200 samples each) cover $\sigma \in \{0, 0.05, 0.1, 0.5, 1\}\%$, read from each file’s `protocol.noise_level`, with 30 equations per level (150 records per system). A record counts as solved if $R^2 \geq$ the threshold recorded in its own file’s protocol block (`protocol.threshold`); the `success` flag is not used (M3 sets it to true whenever no exception is raised).

J.2 Results

Table 24: Solved counts per noise level ($R^2 \geq$ the run’s own recorded threshold; $n = 30$ per cell). Regenerated from the five `protocol_core` files with `generate_tables.py` (`noise_sensitivity.tex`); columns give σ in percent. **Thresholds differ per run** (0.999999, 0.95, 0.95, 0.995, 0.99 for $\sigma = 0, 0.05, 0.1, 0.5, 1\%$; these are the thresholds written into each `protocol_core` file, and differ from the 0.995 that Table 9 applies at $\sigma = 0.05\%$ and 0.1%), so counts are *not* comparable across noise levels, and the pooled column adds counts taken under different thresholds. **Source of the $\sigma = 0$ column:** all six cells come from the noiseless run of the noise sweep, `protocol_core_noiseless_20260812_170344.json`; no cell is taken from `benchmark_results.json` (see the tablenote on Table 4).

Method, σ (%)	0	0.05	0.1	0.5	1	Pooled
EHSDEFI (M3)	23/30	30/30	30/30	30/30	30/30	143/150 (95.3%)
HybridDiscovery v50.2	21/30	25/30	26/30	23/30	26/30	121/150 (80.7%)
HLLMNN (M4)	30/30	30/30	30/30	30/30	30/30	150/150 (100.0%)
ImprovedNN	0/30	29/30	29/30	27/30	29/30	114/150 (76.0%)
PureLLM	30/30	29/30	29/30	30/30	30/30	148/150 (98.7%)
SymbolicEngineWithLLM	28/30	30/30	30/30	29/30	29/30	146/150 (97.3%)

Table 25: Solved counts at a single fixed threshold, $R^2 \geq 0.99$, for every run ($n = 30$ per cell). Because the threshold is the same at all noise levels, counts are comparable across levels and the pooled column is valid. Regenerated with `generate_tables.py --fixed-threshold 0.99 (noise_sensitivity_fixed_0p99.tex)`; columns give σ in percent. The $\sigma = 0$ column is read from `protocol_core_noiseless_20260812_170344.json`, as in Table 24.

Method, σ (%)	0	0.05	0.1	0.5	1	Pooled
EHSD _{EF} I (M3)	30/30	30/30	30/30	30/30	30/30	150/150 (100.0%)
HybridDiscovery v50.2	30/30	24/30	26/30	24/30	26/30	130/150 (86.7%)
HLLMNN (M4)	30/30	30/30	30/30	30/30	30/30	150/150 (100.0%)
ImprovedNN	29/30	27/30	28/30	29/30	29/30	142/150 (94.7%)
PureLLM	30/30	29/30	29/30	30/30	30/30	148/150 (98.7%)
SymbolicEngineWithLLM	30/30	30/30	30/30	30/30	29/30	149/150 (99.3%)

J.3 Key Findings

Ranking. At the fixed $R^2 \geq 0.99$ threshold, M3 and M4 both solve 150/150, ahead of SymbolicEngineWithLLM (149), PureLLM (148), ImprovedNN (142) and HybridDiscovery v50.2 (130). M3’s apparent shortfall in the per-run table (23/30 at $\sigma = 0$) comes from the strict 0.999999 threshold used for that run, not from a loss of accuracy. At that strict threshold M3 solves 23/30 noiseless equations while M4 solves 30/30, so M4 is the only system that is exact under the strict criterion. At 0.99 the two systems cannot be separated.

Threshold confound. The per-run table mixes thresholds from 0.95 to 0.999999. For example ImprovedNN scores 0/30 at $\sigma = 0$ under 0.999999 but 29/30 under 0.99; the jump in the per-run table is the threshold, not the data. Cross-level comparisons should use the fixed-threshold table. The value 0.99 is permissive, so it does not measure exact recovery.

Scope. Only $\sigma \leq 1\%$ is covered, and at $R^2 \geq 0.99$ essentially all systems saturate, so these data do not show where noise starts to hurt. No failure-mode, domain-variability, or noise-threshold claim is supported by these data.